EXPERIMENT NO 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

```
# Module 1 - Create a Blockchain

# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')
```

```
    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash}
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
                                                hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
```

```
        return True

# Part 2 - Mining our Blockchain

# Creating a Web App
app = Flask(__name__)

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200
```

# Running the app
app.run(host = '0.0.0.0', port = 5000)

Output:

127.0.0.1:5000/mine_block

Pretty-print


```
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "fff36e73c543303fb5263617a974feaac234aedcda321bf045e047406579ead7",
  "proof": 533,
  "timestamp": "2026-02-09 02:45:40.792918"
}
```


127.0.0.1:5000/get_chain

Pretty-print


```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-09 02:45:23.070642"
    },
    {
      "index": 2,
      "previous_hash": "fff36e73c543303fb5263617a974feaac234aedcda321bf045e047406579ead7",
      "proof": 533,
      "timestamp": "2026-02-09 02:45:40.792918"
    }
  ],
  "length": 2
}
```

127.0.0.1:5000/get_chain

Pretty-print


```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-09 02:45:23.070642"
    },
    {
      "index": 2,
      "previous_hash": "fff36e73c543303fb5263617a974feaac234aedcda321bf045e047406579ead7",
      "proof": 533,
      "timestamp": "2026-02-09 02:45:40.792918"
    },
    {
      "index": 3,
      "previous_hash": "61804b798fb2f0c906f33996ec398828c292bc1c82a78cf20381bec6f9633697",
      "proof": 45293,
      "timestamp": "2026-02-09 02:49:23.333296"
    },
    {
      "index": 4,
      "previous_hash": "b5fbdc2760830bf879dad4c2d58b387de13ed6c0836e9d27754b923fdf051ebc",
      "proof": 21391,
      "timestamp": "2026-02-09 02:49:30.118351"
    },
    {
      "index": 5,
      "previous_hash": "4853744e72de5b1e2be3a72f0b2e376c15b2905cc82c7981a3cbc347e40bfdbc",
      "proof": 8018,
      "timestamp": "2026-02-09 02:49:35.007604"
    },
```


127.0.0.1:5000/is_valid

Pretty-print


```
{
  "message": "All good. The Blockchain is valid."
}
```


Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.